

IMPLEMENTATION OF ERROR CHECK ALGORITHMS USING PYTHON

Members: Ayush Shukla-200907282 Sandali Singh-200907044 ; Aaryan Alexander Martis-200907256 ;

This report provides a comprehensive overview of essential error detection techniques and their practical implementations in Python. By utilizing an interactive approach, the report aims to enhance the reader's understanding of how these techniques can be applied in real-world scenarios to ensure data accuracy and integrity during transmission over unreliable channels.

Error control and check:

Error control and checking are crucial in communication systems to ensure the accuracy and reliability of transmitted data. In any communication process, especially in digital data transmission, errors can occur due to various factors such as noise, interference, distortion, or hardware/software malfunctions. Implementing error control and checking mechanisms helps in the detection, correction, and prevention of these errors, thus ensuring the integrity of the data being transmitted.

We can make use of various standard algorithms for this, some of which are:

1. CRC
2. Checksum
3. Parity Check

Hereby follows the explanation and working of the listed algorithms.

Cyclic Redundancy Check (CRC)

CRC, which stands for Cyclic Redundancy Check, is an error-detection technique commonly used in digital networks and storage devices to detect accidental changes to raw data. It works by generating a checksum, which is a fixed-size bit string, based on the data that is being transmitted. The checksum is appended to the data, and the recipient can use it to verify the integrity of the received data.

The working principle of CRC can be summarized as follows:

1. **Message and Polynomial Representation:** The data is represented as a sequence of bits, with the most significant bit typically being the leftmost bit. The CRC algorithm uses a predefined polynomial, known as the generator polynomial, which is selected based on the specific application requirements. The polynomial is also represented as a sequence of bits.
2. **Padding with Zeros:** Before the CRC calculation, the message is padded with zeros at the end to match the length of the generator polynomial.
3. **Division and Remainder Calculation:** The padded message is divided by the generator polynomial using binary division. The remainder obtained from this division is the CRC.
4. **Appending the CRC:** The CRC is then appended to the original message, creating a new bit string that is transmitted along with the data.
5. **Error Detection at Receiver's End:** Upon receiving the data, the recipient repeats the division process. If the remainder obtained is zero, it indicates that the data has been received without any errors. However, if the remainder is nonzero, it implies that errors may have occurred during transmission.

The key advantage of CRC is its effectiveness in detecting a variety of errors, such as single-bit errors and most common multiple-bit errors. However, it is important to note that CRC does not provide error correction capabilities; it can only detect errors. Additionally, the efficiency of CRC depends on the chosen generator polynomial, as certain polynomials may be more suitable for specific types of data and applications.

CRC is widely used in various communication protocols, such as Ethernet, USB, and SATA, to ensure the accuracy and reliability of data transmission. Its simplicity, efficiency, and versatility have made it one of the most popular error-detection techniques in digital communication systems.

Working:

Let's say we want to send the following 4-bit message: 1101.

First, we choose a generator polynomial, such as $G(x) = x^3 + x + 1$.

You perform the CRC calculation.

a. Append three zeros to the message to make it 1101000.

b. Perform a modulo-2 division.

$1101000 / 1011 = 10100$ with a remainder of 101.

Replace the appended zeros with the remainder, giving you the final transmitted message: 1101101.

At the receiver's end, the same CRC polynomial is used to check for errors. The received message is divided by the same polynomial. If the remainder is non-zero, an error is detected.

Checksum

A checksum is a simple form of error-detecting code in a data transmission or storage system. It involves the addition of a sequence of numbers or characters that are calculated from the data packet. The purpose of the checksum is to enable error detection by verifying the integrity of the data.

The checksum calculation involves the following steps:

1. A sequence of data, such as binary digits, characters, or numbers, is divided into fixed-size segments.
2. The individual segments are then added together to create a sum.
3. The sum is often complemented or manipulated in some other way to create a checksum value.
4. This checksum value is transmitted or stored along with the original data.

At the receiving end, the same process is repeated on the received data, and the computed checksum is compared with the transmitted checksum. If the two checksums match, it suggests that the data is likely to be error-free. If the checksums do not match, an error is detected, and the data may need to be retransmitted or corrected.

Checksums are commonly used in various applications such as data transmission over networks, storage systems, and digital communication protocols to ensure data integrity and to detect transmission errors, data corruption, or tampering.

Working:

Suppose we want to send a data consisting of (decimal) : 3,6,2,1,4.

To create a checksum for this set, you can follow these steps:

1. Add all the numbers together: $3 + 6 + 2 + 1 + 4 = 16$.
2. So, in this example, the checksum for the set of numbers 3, 6, 2, 1, 4 is 16.

At the receiving end, the receiver performs the same calculation on the received data. If the calculated checksum does not match the transmitted checksum, it suggests that an error might have occurred during the transmission of the data. This way, checksums help in verifying the integrity of the data.

Parity Check

Parity check is a simple method used for the detection of errors in data transmission. It involves the addition of a single bit to a data unit to make the number of ones either even or odd. The purpose of this additional bit is to enable the receiver to determine whether the data received is accurate.

There are two types of parity checks:

Even Parity: In this method, the number of 1s in the data unit, including the parity bit, is counted. The parity bit is set to 0 or 1 to make the total count of 1s even.

Odd Parity: In this method, the number of 1s in the data unit, including the parity bit, is counted. The parity bit is set to 0 or 1 to make the total count of 1s odd.

Working:

Suppose we want to transmit the binary data 1011 using even parity.

Count the number of 1s in the data unit: $1 + 0 + 1 + 1 = 3$.

As the count of 1s is odd, the even parity bit is added as 1, making the data unit and parity bit 10111.

At the receiving end, the receiver counts the number of 1s, including the parity bit.

If the count is even as expected, the data is error-free.

If the count is odd, an error is detected, indicating that the data might have been corrupted during transmission. Parity checks are relatively simple and are commonly used in situations where a basic form of error detection is sufficient.